

BIT Platform — AI Engine Architecture & Implementation

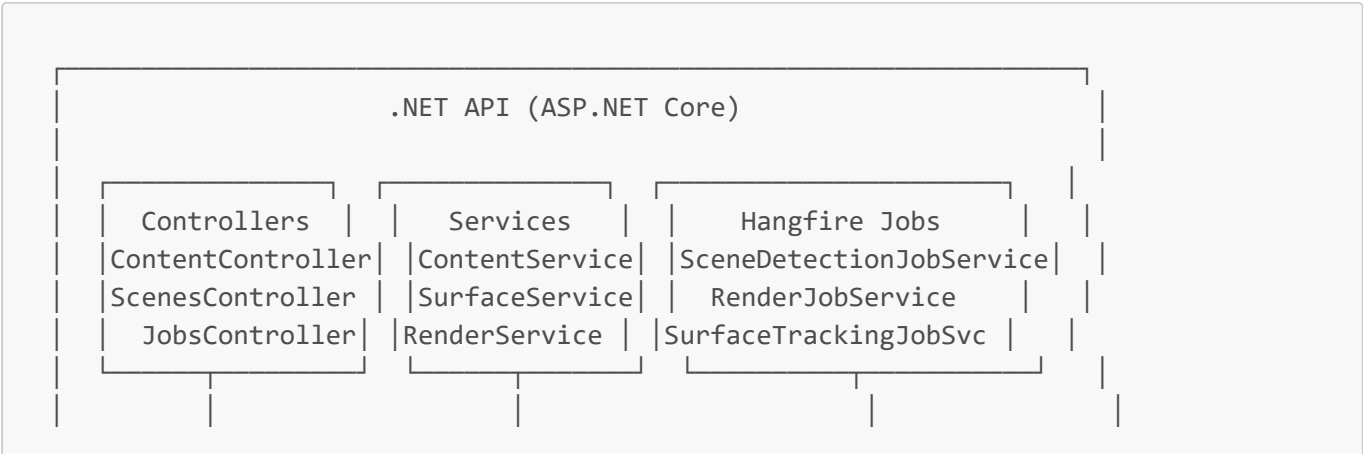
Version: 3.0
Date: 2026-07-25
Status: Production

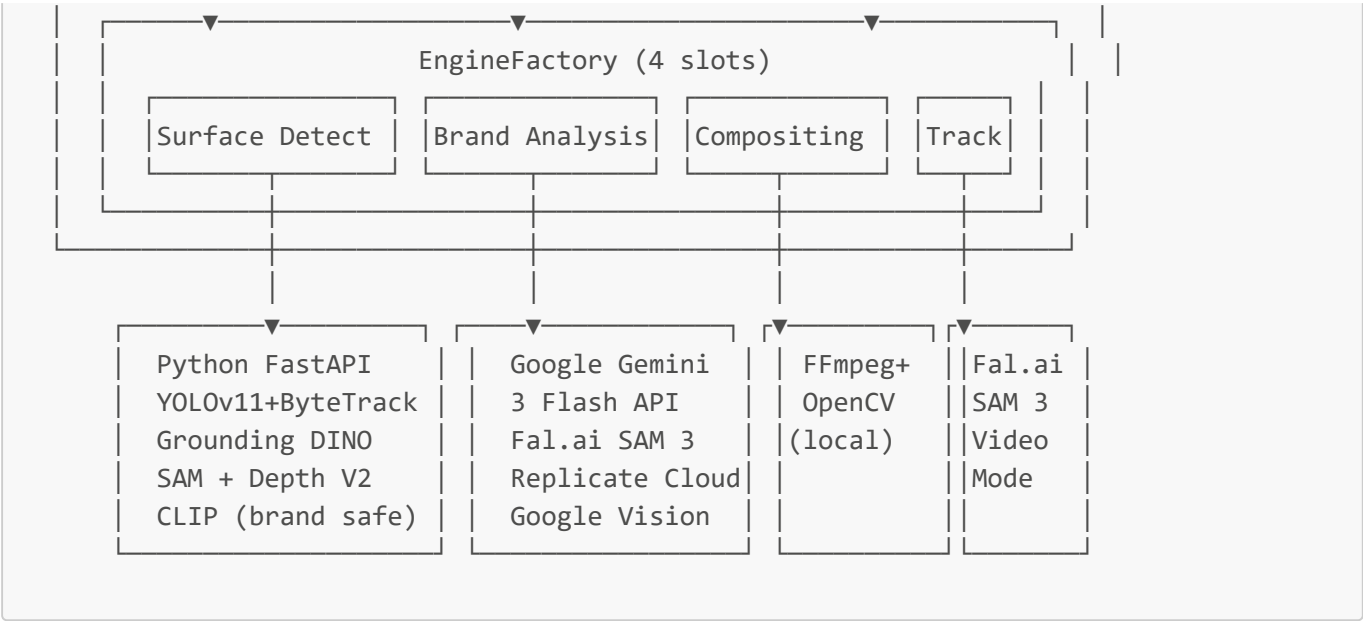
Engine Version Summary (Post-Upgrade)

Slot	Engine	Version	Status
Surface Detection	Gemini Detection	3 Flash	✓ Active
Surface Detection	YOLO	v11 + ByteTrack	✓ Active
Surface Detection	Grounding DINO v2	GD + SAM + Depth + CLIP	✓ Active
Surface Detection	Replicate	SAM 3 (single call)	✓ Active
Surface Detection	Google Vision	Cloud Vision API	✓ Active
Mask Refinement	Fal.ai SAM 3	Image	✓ Active
Brand Analysis	Gemini 3 Flash	Multimodal	✓ Real
Brand Analysis	Google Vision	Logo + Text + Label	✓ Real
Compositing	FFmpeg/OpenCV	Overlay + Blend	✓ Active
Surface Tracking	SAM 3 Video	Per-frame tracking	NEW Phase 3
Render Stitching	FFmpeg concat	Real libx264 encode	✓ Real

1. Architecture Overview

The BIT platform uses a **pluggable AI engine architecture** with four independently swappable engine slots, each resolved at runtime via a central **EngineFactory**. All engines implement a common interface, and the active engine is determined by database-backed **Platform Settings** (with `appsettings.json` fallbacks). This allows operators to swap engines without redeploying the .NET API.





Key Design Principles

- 1. **No mock code ever** — The `Basic*Service` classes explicitly throw or return empty results to force admin configuration of real engines (governance rule: `no-mock-code.md`).
- 2. **Factory pattern with async resolution** — `EngineFactory` queries `PlatformSettingsService` (DB-backed) on every resolution, allowing runtime engine swaps.
- 3. **DI container never blocks** — Engine resolution via `IServiceProvider` scope creation prevents startup deadlocks.
- 4. **All engines implement a common interface** — `ISurfaceDetectionService`, `IBrandAnalysisService`, `ICompositingService`, `ISurfaceTrackingService`.

2. Engine Factory (`EngineFactory.cs`)

File: `dotnet-api/Services/EngineFactory.cs`

Interface: `IEngineFactory`

The factory resolves four independent engine slots based on platform setting keys:

Slot	Setting Key	Default	Interface
Surface Detection	<code>engine_detection</code>	"basic"	<code>ISurfaceDetectionService</code>
Brand Analysis	<code>engine_brand_analysis</code>	"basic"	<code>IBrandAnalysisService</code>
Compositing	<code>engine_compositing</code>	"opencv"	<code>ICompositingService</code>
Surface Tracking	<code>engine_tracking</code>	"basic"	<code>ISurfaceTrackingService</code>

Surface Detection Engine Resolution

Setting Value	Engine Class	Type	Requires
"yolo"	<code>YoloSurfaceDetectionService</code>	Local Python	Python FastAPI on port 8001

Setting Value	Engine Class	Type	Requires
"grounding-dino"	GroundingDinoDetectionService	Local Python	Python FastAPI on port 8001 (v2 endpoint)
"gemini"	GeminiDetectionService	Cloud API	gemini_api_key setting
"google"	GoogleVisionDetectionService	Cloud API	google_vision_api_key setting
"replicate"	ReplicateSurfaceDetectionService	Cloud API	replicate_api_key setting
"basic" (default)	BasicSurfaceDetectionService	N/A	Throws — forces admin config

Brand Analysis Engine Resolution

Setting Value	Engine Class	Type
"google"	GoogleVisionBrandAnalysisService	Cloud API
"gemini"	GeminiBrandAnalysisService	Cloud API
"basic" (default)	BasicBrandAnalysisService	No-op (returns empty)

Compositing Engine Resolution

Setting Value	Engine Class	Type
"opencv"	OpenCvCompositingService	Local FFmpeg
"basic" (default)	BasicCompositingService	Returns asset image as-is

DI Registration (Program.cs)

```
// Each engine registered individually
builder.Services.AddScoped<GeminiDetectionService>();
builder.Services.AddScoped<FalAiSam3Service>();
builder.Services.AddScoped<ReplicateSurfaceDetectionService>();
builder.Services.AddScoped<GoogleVisionDetectionService>();
builder.Services.AddScoped<YoloSurfaceDetectionService>();
builder.Services.AddScoped<GroundingDinoDetectionService>();
builder.Services.AddScoped<BasicSurfaceDetectionService>();
builder.Services.AddScoped<GoogleVisionBrandAnalysisService>();
builder.Services.AddScoped<GeminiBrandAnalysisService>();
builder.Services.AddScoped<BasicBrandAnalysisService>();
builder.Services.AddScoped<OpenCvCompositingService>();
builder.Services.AddScoped<BasicCompositingService>();

// Phase 3: Surface tracking engine
builder.Services.AddScoped<Sam3TrackingService>();
```

```

builder.Services.AddScoped<BasicTrackingService>();

// Hangfire job services
builder.Services.AddScoped<RenderJobService>();
builder.Services.AddScoped<SceneDetectionJobService>();
builder.Services.AddScoped<SurfaceTrackingJobService>();

// Factory
builder.Services.AddScoped<IEngineFactory, EngineFactory>();

// Interface resolution → delegates to factory at DI time
builder.Services.AddScoped<ISurfaceDetectionService>(sp =>
    sp.GetRequiredService<IEngineFactory>
    ().GetSurfaceDetectionEngineAsync().GetAwaiter().GetResult());
builder.Services.AddScoped<IBrandAnalysisService>(sp =>
    sp.GetRequiredService<IEngineFactory>
    ().GetBrandAnalysisEngineAsync().GetAwaiter().GetResult());
builder.Services.AddScoped<ICompositingService>(sp =>
    sp.GetRequiredService<IEngineFactory>
    ().GetCompositingEngineAsync().GetAwaiter().GetResult());
builder.Services.AddScoped<ISurfaceTrackingService>(sp =>
    sp.GetRequiredService<IEngineFactory>
    ().GetTrackingEngineAsync().GetAwaiter().GetResult());

```

3. Surface Detection Engines (Detailed)

All surface detection engines implement `ISurfaceDetectionService`:

```

public interface ISurfaceDetectionService
{
    Task<List<SurfaceDetectionResult>> DetectAsync(
        string contentId, int sceneIndex, int startFrame, int endFrame,
        CancellationToken cancellationToken = default);

    // Batch detection (engines that support it override for performance)
    Task<List<SceneDetectionBatchResult>> DetectBatchAsync(
        string contentId, string videoPath, List<SceneCut> scenes,
        CancellationToken cancellationToken = default);
}

```

Result Model

```

public class SurfaceDetectionResult
{
    public string SurfaceType { get; set; } // e.g. "Billboard", "TV
    Screen"
    public string BoundaryCoordinatesJson { get; set; } // JSON array of {x,y}
    polygon points
}

```

```
public double EstimatedDepth { get; set; } // metres
public string OrientationVectorJson { get; set; } // {yaw, pitch, roll}
public double ConfidenceScore { get; set; } // 0.0-1.0
public double ViabilityScore { get; set; } // 0.0-1.0 (composite)
public string? ExclusionReason { get; set; } // null = safe, string =
auto-rejected
}
```

3a. YOLOv11 + ByteTrack (YoloSurfaceDetectionService)

Activation: engine_detection = "yolo"
Endpoint: POST http://localhost:8001/detect
Timeout: 15 minutes
Health check: 3 retries × 2s backoff against /health

Pipeline (Python side):

- 1. Open video file with OpenCV
- 2. Run YOLOv11 inference on each frame (or every Nth frame per frame_skip)
- 3. ByteTrack assigns stable track IDs across frames
- 4. Surfaces matched to COCO classes: tvmonitor (62), laptop (63), cell phone (67), book (73), clock (74), keyboard (76)
- 5. Aspect-ratio heuristic classifies into BIT surface types:
 - 2.5: "Stadium Perimeter LED Board"
 - 1.5: "Digital Screen"
 - 1.3: "Signage Panel"
 - Default: "TV Screen"
- 6. Person class (COCO 0) permanently excluded (brand safety)
- 7. Composite viability = confidence × bounding-box size × aspect-ratio fit

Configurable Platform Settings:

Key	Default	Description
yolo_service_url	http://localhost:8001	Python service URL
yolo_model_size	"large"	nano/small/medium/large/xlarge
yolo_confidence	0.35	Detection confidence threshold
yolo_iou	0.45	NMS IoU threshold
yolo_frame_skip	1	Process every Nth frame (1=100%)

Limitation: YOLO detects only COCO-pretrained classes (80 types). Cannot detect novel surfaces like "empty brick wall", "bus side", "stadium LED board" unless those objects happen to correlate with a COCO class. For open-vocabulary detection, use grounding-dino.

3b. Grounding DINO v2 (GroundingDinoDetectionService)

Activation: engine_detection = "grounding-dino"
Endpoint: POST http://localhost:8001/detect-v2
Timeout: 10 minutes

Pipeline (Python side — engine_v2.py):

- 1. **Adaptive frame-skip** (optical flow): Only runs detection when scene motion exceeds flow_motion_threshold (2.5 px mean), skips up to max_frame_skip (15) frames between detections
- 2. **Grounding DINO** (zero-shot): Accepts a text prompt describing surface characteristics (not fixed categories). Detects any surface matching the description — billboards, walls, bus sides, stadium boards, windows, etc.
- 3. **IoU-based tracking** with re-identification: Tracks surfaces across frames with configurable lost-buffer (30 frames). Surfaces seen in fewer than track_min_detection_frames (3) frames are filtered out.
- 4. **SAM** (Segment Anything Model): Generates pixel-perfect polygon masks from bounding boxes (optional, toggle: gd_enable_sam)
- 5. **Depth Anything V2**: Per-surface real depth estimation from mask regions (optional, toggle: gd_enable_depth)
- 6. **CLIP**: Zero-shot brand-safety classification — auto-rejects unsafe surfaces (optional, toggle: gd_enable_brand_safety)

Configurable Platform Settings:

Key	Default	Description
gd_model_variant	"base"	base / tiny
gd_box_threshold	0.25	Detection box threshold
gd_text_threshold	0.20	Text-prompt matching threshold
gd_enable_sam	true	Enable SAM polygon segmentation
gd_enable_depth	true	Enable Depth Anything V2
gd_enable_brand_safety	true	Enable CLIP brand-safety
gd_enable_tracking	true	Enable multi-frame tracking
gd_adaptive_frame_skip	true	Optical-flow-based frame skipping
gd_detection_interval	10	Run detection every N frames
gd_flow_motion_threshold	2.5	Mean flow magnitude to trigger
gd_track_min_frames	3	Min frames for valid track

3c. Gemini 3 Flash (GeminiDetectionService)

Activation: engine_detection = "gemini"
Endpoint: POST https://generativelanguage.googleapis.com/v1beta/models/gemini-3-

flash:generateContent

Cost: ~\$0.0001/image

Timeout: Configurable via `gemini_timeout_seconds` setting (default 90s). Bumped from 45s — generous enough to survive Google rate-limit backoff, large 4K broadcast key-frames, and production network jitter. Load-test under real conditions before lowering. **JSON Mode:** Uses `response_mime_type: "application/json"` for native structured output. Markdown fence stripping retained as fallback.

Pipeline:

1. Extract key frame (middle frame of scene) as base64 JPEG via FFmpeg
2. Send to Gemini 3 Flash with a structured multimodal prompt + native JSON mode (`response_mime_type: "application/json"`)
3. **Single API call replaces four separate models:**
 - Zero-shot surface detection (like Grounding DINO)
 - Boundary polygon estimation (like SAM)
 - Brand-safety classification (like CLIP)
 - Surface type classification

Prompt Design: The prompt instructs Gemini to find *every* potentially ad-placeable surface with no category limits. It covers walls, ceilings, floors, vehicle exteriors, windows, doors, tables, clothing, packaging, natural surfaces, electronic displays, signs, furniture — ANY flat or semi-flat visible region.

Brand Safety (in-prompt): The prompt permanently excludes:

- Human faces, heads, bodies
- Children/minors
- Emergency vehicles
- Military/weapons
- Religious symbols/places
- Government buildings/insignia
- Alcohol/tobacco/drugs
- Gore/violence/explicit content

Returns max 20 surfaces, sorted by viability descending.

Configurable Settings:

Key	Default
<code>gemini_api_key</code>	(required)
<code>gemini_model</code>	<code>gemini-3-flash</code>
<code>gemini_timeout_seconds</code>	90

3d. Fal.ai SAM 3 (`FalAiSam3Service`)

Used by: Gemini pipeline (post-processing)

Endpoint: POST <https://fal.run/fal-ai/sam-3/image>

Cost: ~\$0.001 per mask call

Timeout: 3 minutes

Pipeline:

1. Takes bounding boxes + surface type labels from Gemini detection results
2. Sends image + boxes + text prompts to Fal.ai SAM 3 API (combined prompting for higher accuracy)
3. Receives multiple mask candidates with per-instance confidence scores; takes top-scored mask per box
4. Falls back to bounding-box quads if API key not configured or API call fails

Configurable Settings (Admin UI):

Key	Default
<code>falai_api_key</code>	(required)
<code>falai_sam3_endpoint</code>	<code>https://fal.run/fal-ai/sam-3/image</code>
<code>falai_sam2_endpoint</code>	(deprecated — kept for rollback)

3e. Replicate Cloud — SAM 3 (`ReplicateSurfaceDetectionService`)

Activation: `engine_detection = "replicate"`

Timeout: 5 minutes

Pipeline:

1. Extract key frame as base64 via FFmpeg
2. **Single SAM 3 call** (`lucataco/sam3-image` on Replicate) — detects surfaces + generates polygon masks in one API call (replaces old 2-call GD+SAM2 pipeline)
3. Passes open-vocabulary text prompt directly as SAM 3's concept prompt
4. Polls Replicate prediction until completion, parses masks + labels + scores

Text Prompt: "a flat rectangular surface . a smooth empty area . a visible wall or panel . a screen or display . a large flat side of an object..."

Configurable Settings:

Key	Default
<code>replicate_api_key</code>	(required)
<code>replicate_sam3_model</code>	<code>lucataco/sam3-image</code>
<code>replicate_sam3_version</code>	(required — pin this)
<code>replicate_gd_model</code>	(deprecated — kept for rollback)
<code>replicate_sam_model</code>	(deprecated — kept for rollback)
<code>replicate_box_threshold</code>	(model default)
<code>replicate_text_threshold</code>	(model default)

⚠️ `lucataco/sam3-image` is a **community-published model**, not an official Meta-maintained one. The owner can update, retag, or deprecate it at any time. Set `replicate_sam3_version` to an explicit version hash to prevent silent upstream changes from breaking your output schema. Check the model page on Replicate periodically for maintenance status.

3f. Google Cloud Vision (`GoogleVisionDetectionService`)

Activation: `engine_detection = "google"`

Endpoint: `POST https://vision.googleapis.com/v1/images:annotate`

Timeout: 2 minutes

Pipeline:

1. Extract key frame as base64
 2. Call Vision API with `OBJECT_LOCALIZATION` feature (max 50 results)
 3. Filter results against a taxonomy of ad-placeable surface labels:
 - billboard, signage, sign, poster, banner
 - television, display device, screen, monitor, LED display
 - advertisement, brand, logo
 - wall, building, window, bus, vehicle, truck
 - scoreboard, stadium, arena
 4. Map bounding boxes to BIT surface candidates
-

3g. Basic (`BasicSurfaceDetectionService`)

Activation: `engine_detection = "basic"` (default)

Behavior: Throws `InvalidOperationException` with the message:

```
"No AI detection engine is configured. Set Platform Setting 'engine_detection' to 'yolo', 'grounding-dino', 'replicate', 'gemini', or 'google'."
```

This is intentional — forces operators to configure a real engine. Per governance rule `no-mock-code.md`, random mock data is NEVER acceptable.

4. Brand Analysis Engines

All implement `IBrandAnalysisService`:

```
public interface IBrandAnalysisService
{
    Task<BrandAnalysisResult> AnalyzeAsync(
        string contentId, string surfaceType, string frameRegionBase64);
}

public class BrandAnalysisResult
{
    public List<string> DetectedBrands { get; set; }
```

```
public List<string> DetectedLogos { get; set; }
public List<string> DetectedText { get; set; }
public bool HasCompetitiveConflict { get; set; }
public string? ConflictDescription { get; set; }
public double ConfidenceScore { get; set; }
}
```

Engine	Status	Description
GoogleVisionBrandAnalysisService	✓ Real	Vision API: LOGO_DETECTION + TEXT_DETECTION + LABEL_DETECTION. Returns brands, logos, text with confidence scores.
GeminiBrandAnalysisService	✓ Real	Gemini 3 Flash: structured JSON prompt for brand, logo, text, and competitive conflict detection.
BasicBrandAnalysisService	✓ Default	Returns empty BrandAnalysisResult — no-op until a real engine is configured.

5. Brand Safety Check Pipeline

Service: BrandSafetyCheckService (implements IBrandSafetyCheckService)

Runs between surface detection and persistence. Checks every detected surface against:

Permanent exclusions (hardcoded):

- Human Faces
- Children
- Emergency Vehicles
- Government Insignia
- Religious Symbols
- Religious Spaces

Database-driven exclusions: Active BrandSafetyRules from the DB (BrandSafetyRules table, filtered by IsActive = true)

Matching logic: Case-insensitive substring matching of surface type against exclusion category names, with special handling for faces, children, and religious content.

6. Compositing Engines

All implement ICompositingService:

```
public interface ICompositingService
{
    Task<CompositedFrame> CompositeAsync(CompositingRequest request);
}
```

6a. FFmpeg/OpenCV (**OpenCvCompositingService**)

Activation: `engine_compositing = "opencv"`

Pipeline:

- 1. Resolve asset file from `CreativeAssets.StorageKey`
- 2. Resolve video file from `ContentItems.StorageKey`
- 3. Determine capture frame from the surface's scene start frame
- 4. Parse boundary coordinates JSON to get overlay position (`x, y, w, h`)
- 5. Extract video frame via FFmpeg `select` filter (falls back to `-ss` time-seek if frame-accurate extraction fails)
- 6. Overlay brand asset onto frame at the surface position using FFmpeg `overlay` filter
- 7. Return composited frame as base64 PNG

Fallback: If compositing fails at any step, falls back to `BasicCompositingService`.

6b. Basic (**BasicCompositingService**)

Activation: `engine_compositing = "basic"` (default)

Returns the asset image as-is (no actual compositing). Reads asset file from disk and returns it as base64. This is the placeholder that forces admin to configure a real compositing engine.

7. Unified Detection Pipeline (**SurfaceDetectionPipeline**)

File: `dotnet-api/Services/SurfaceDetectionPipeline.cs`

Orchestrates the full detection flow as a Hangfire background job. Has three entry points:

Method	Hangfire Job	Description
<code>RunAsync()</code>	<code>RunDetectionPipeline</code>	Full: FFmpeg scenes → Gemini → SAM3 → persist
<code>RunScenesOnlyAsync()</code>	<code>RunScenesOnlyPipeline</code>	Cheap: FFmpeg scenes → thumbnails only
<code>RunSurfaceDetectionForSceneAsync()</code>	<code>RunSceneSurfaceDetection</code>	Per-scene: Gemini → SAM3 for one scene

Pipeline Phases (**RunAsync**)

Progress	Step
5%	Transition to SceneDetecting stage
10%	FFmpeg scene cut detection
40%	Delete old scenes & surfaces
42-82%	For each scene: 44% Extract key frame + call Gemini

	48%	Persist scene + surfaces (progress scaled by scene index)
90%		Generate thumbnails for each scene
100%		Complete – set status to Completed

Pause/Resume Support

Added 2026-07-25: Between each scene iteration, the pipeline calls `WaitIfPaused()` which:

- 1. Queries `ContentItems.IsDetectionPaused` from DB
- 2. If paused: sets `JobState = "Paused"`, sleeps 3s, polls again
- 3. If unpaused: sets `JobState = "Processing"`, continues
- 4. Respects Hangfire `CancellationToken` for job cancellation

8. Python Detection Service

File: `detection-service/main.py`
Port: 8001 (default)
Framework: FastAPI
Version: 2.0.0

Endpoints

Method	Path	Engine	Description
GET	<code>/health</code>	Both	Health check — returns <code>{"status":"ok","model_loaded":true/false}</code>
POST	<code>/detect</code>	v1 (YOLO)	Per-scene YOLOv11 + ByteTrack detection
POST	<code>/detect-batch</code>	v1 (YOLO)	Batch detection across multiple scenes
POST	<code>/detect-v2</code>	v2 (GD+SAM+Depth+CLIP)	Full open-vocabulary pipeline

v1: YOLO + ByteTrack (`detector.py`)

- **Model:** YOLOv11 (`ultralytics`) — nano (`yolo11n.pt`) or large (`yolo11l.pt`) per config
- **Classes:** 6 COCO classes mapped to ad surfaces (tvmonitor, laptop, cell phone, book, clock, keyboard)
- **Tracking:** ByteTrack for frame-to-frame ID consistency
- **Brand safety:** Person (COCO 0) permanently excluded
- **Classification:** Aspect-ratio heuristic maps bounding-box shapes to BIT surface types

v2: Grounding DINO + SAM + Depth + CLIP (`engine_v2.py`)

- **Grounding DINO:** Zero-shot detection from text prompt (open vocabulary — no class limits)
- **SAM:** Precise polygon masks for every detection
- **Depth Anything V2:** Per-surface real depth estimation

- **CLIP:** Brand-safety auto-classification
- **Tracking:** IoU-based multi-frame tracking with re-identification
- **Adaptive frame-skip:** Optical flow analysis skips static frames, runs detection on motion

Module files:

File	Purpose
main.py	FastAPI server, request/response models
detector.py	YOLO detector + ByteTrack + aspect-ratio classification
engine_v2.py	v2 pipeline orchestrator (GD + SAM + Depth + CLIP + tracking)
grounding_dino_detector.py	Grounding DINO zero-shot detector
sam_segementer.py	SAM polygon mask generator
depth_estimator.py	Depth Anything V2 estimator
brand_safety.py	CLIP brand-safety classifier
tracker.py	IoU-based multi-frame surface tracker
requirements.txt	Python dependencies
yolo11l.pt	YOLOv11 large weights
yolo11n.pt	YOLOv11 nano weights

9. Render Engine (RenderJobService)

File: dotnet-api/Services/RenderJobService.cs

Hangfire background job for compositing and video stitching.

Pipeline

Progress	Phase
5-25%	Asset Validation – lookup content, surface, scene, asset; verify video file exists
25-60%	Compositing – call ICompositingService to overlay asset onto video frame at surface position
60-100%	FFmpeg Video Generation – extract scene frames, overlay composited asset, encode with libx264

Progress reflects actual ICompositingService work and FFmpeg encoding time. No simulated delays. Output is a real MP4 file served via `/api/renders/{id}/download`.

10. Platform Settings Reference

All engine configuration is stored in the `PlatformSettings` DB table, with `appsettings.json` fallbacks. Editable via the Admin Console UI.

Key	Category	Default	Description
<code>engine_detection</code>	Engine	<code>basic</code>	Surface detection engine: yolo, grounding-dino, gemini, google, replicate
<code>engine_brand_analysis</code>	Engine	<code>basic</code>	Brand analysis engine: google, gemini
<code>engine_compositing</code>	Engine	<code>opencv</code>	Compositing engine: opencv
<code>engine_tracking</code>	Engine	<code>basic</code>	Surface tracking engine: sam3
<code>gemini_api_key</code>	Engine	—	Google Gemini API key
<code>falai_api_key</code>	Engine	—	Fal.ai API key (SAM 3 image + video)
<code>replicate_api_key</code>	Engine	—	Replicate API key (SAM 3 cloud)
<code>google_vision_api_key</code>	Engine	—	Google Cloud Vision API key
<code>gemini_model</code>	Engine	<code>gemini-3-flash</code>	Gemini model identifier
<code>gemini_timeout_seconds</code>	Engine	<code>90</code>	Per-request HTTP timeout (survive rate-limit backoff + jitter)
<code>falai_sam3_endpoint</code>	Engine	<code>https://fal.run/fal-ai/sam-3/image</code>	SAM 3 image endpoint
<code>sam3_tracking_endpoint</code>	Engine	<code>https://fal.run/fal-ai/sam-3/video</code>	SAM 3 video tracking endpoint
<code>replicate_sam3_model</code>	Engine	<code>lucataco/sam3-image</code>	Replicate SAM 3 model slug (community model — pin version)
<code>replicate_sam3_version</code>	Engine	—	Version hash to pin model (prevent silent upstream changes)
<code>replicate_gd_model</code>	Engine	(deprecated)	Old Grounding DINO model — rollback only
<code>replicate_sam_model</code>	Engine	(deprecated)	Old SAM 2 model — rollback only

Key	Category	Default	Description
yolo_service_url	Engine	http://localhost:8001	Python detection service URL
yolo_model_size	Engine	large	YOLO model variant
yolo_confidence	Engine	0.35	Detection confidence threshold
yolo_iou	Engine	0.45	NMS IoU threshold
yolo_frame_skip	Engine	1	Process every Nth frame
gd_model_variant	Engine	base	Grounding DINO variant
gd_box_threshold	Engine	0.25	GD box threshold
gd_text_threshold	Engine	0.20	GD text threshold
gd_enable_sam	Engine	true	Enable SAM segmentation
gd_enable_depth	Engine	true	Enable Depth Anything V2
gd_enable_brand_safety	Engine	true	Enable CLIP brand safety
gd_enable_tracking	Engine	true	Enable multi-frame tracking
gd_adaptive_frame_skip	Engine	true	Optical-flow-based frame skipping
gd_detection_interval	Engine	10	Detection interval in frames
gd_flow_motion_threshold	Engine	2.5	Motion trigger threshold
gd_track_min_frames	Engine	3	Min frames for valid track
smtp_host	SMTP	—	SMTP server hostname
smtp_port	SMTP	587	SMTP server port
smtp_user	SMTP	—	SMTP username
smtp_password	SMTP	—	SMTP password
smtp_from_email	SMTP	—	From email address
upload_max_video_bytes	Upload	10737418240	Max video upload (bytes)
upload_max_asset_bytes	Upload	104857600	Max asset upload (bytes)
fps_min	Pipeline	1	Minimum frame rate
fps_max	Pipeline	960	Maximum frame rate
scene_detect_threshold	Pipeline	0.4	FFmpeg scene change threshold

Key	Category	Default	Description
idle_timeout_minutes	Session	28	Session idle timeout
idle_countdown_seconds	Session	60	Idle countdown warning
jwt_expiry_hours	Auth	8	JWT token lifetime
jwt_refresh_window_hours	Auth	2	JWT refresh window

11. Engine Comparison Matrix

Feature	YOLOv11	Grounding DINO v2	Gemini 3 Flash	Replicate SAM 3	Google Vision
Detection type	Fixed 80 COCO classes	Open-vocabulary (any text)	Multimodal (any visual)	Open-vocabulary	1000+ labels
Polygon masks	Bounding boxes only	SAM masks	Gemini polygons	SAM masks	Bounding boxes only
Depth estimation	Heuristic (bbox size)	Depth Anything V2	Gemini-estimated	Heuristic	None
Brand safety	Person exclusion only	CLIP classification	In-prompt exclusions	None	None
Multi-frame tracking	ByteTrack	IoU + re-ID	Per-frame (stateless)	Per-frame	Per-frame
Adaptive frame-skip	Configurable skip	Optical flow	N/A	N/A	N/A
GPU required	Yes (local)	Yes (local)	No (cloud)	No (cloud)	No (cloud)
Cost per frame	Free (local HW)	Free (local HW)	~\$0.0001	~\$0.003	~\$0.0015
Latency per scene	1–5s (GPU)	2–10s (GPU)	0.5–2s	3–15s	0.5–2s
Setup complexity	Medium (Python + CUDA)	High (4 models + CUDA)	Low (API key only)	Low (API key only)	Low (API key only)
Production readiness	✔ High	✔ High	✔ High	● Medium	● Medium

12. Source File Index

File	Purpose
<code>dotnet-api/Services/EngineFactory.cs</code>	Central engine resolution (4 slots) from Platform Settings
<code>dotnet-api/Services/IEngineFactory.cs</code>	Factory interface (4 slots)
<code>dotnet-api/Services/ISurfaceDetectionService.cs</code>	Detection interface + result models
<code>dotnet-api/Services/IBrandAnalysisService.cs</code>	Brand analysis interface + result model
<code>dotnet-api/Services/ICompositingService.cs</code>	Compositing interface
<code>dotnet-api/Services/ISurfaceTrackingService.cs</code>	 Tracking interface + FrameBoundary model
<code>dotnet-api/Services/YoloSurfaceDetectionService.cs</code>	YOLO v1 engine — calls Python service
<code>dotnet-api/Services/GroundingDinoDetectionService.cs</code>	Grounding DINO v2 engine — calls Python service
<code>dotnet-api/Services/GeminiDetectionService.cs</code>	Gemini 3 Flash engine — cloud API with JSON mode
<code>dotnet-api/Services/FalAiSam3Service.cs</code>	 Fal.ai SAM 3 — mask refinement with combined box+text
<code>dotnet-api/Services/FalAiSam2Service.cs</code>	 Deprecated — replaced by SAM 3, kept for rollback
<code>dotnet-api/Services/ReplicateSurfaceDetectionService.cs</code>	Replicate SAM 3 engine — single-call detection+segmentation
<code>dotnet-api/Services/GoogleVisionDetectionService.cs</code>	Google Cloud Vision engine
<code>dotnet-api/Services/BasicSurfaceDetectionService.cs</code>	Placeholder — throws to force config
<code>dotnet-api/Services/GoogleVisionBrandAnalysisService.cs</code>	 Real — Vision API logo+text+label detection
<code>dotnet-api/Services/GeminiBrandAnalysisService.cs</code>	 Real — Gemini 3 Flash brand/logo/conflict analysis
<code>dotnet-api/Services/BasicBrandAnalysisService.cs</code>	No-op brand analysis
<code>dotnet-api/Services/BrandSafetyCheckService.cs</code>	Brand-safety filter — permanent + DB rules
<code>dotnet-api/Services/OpenCvCompositingService.cs</code>	FFmpeg-based compositing
<code>dotnet-api/Services/BasicCompositingService.cs</code>	Placeholder compositing
<code>dotnet-api/Services/Sam3TrackingService.cs</code>	 SAM 3 video mode — full-scene per-frame tracking

File	Purpose
<code>dotnet-api/Services/BasicTrackingService.cs</code>	 Placeholder — throws to force config
<code>dotnet-api/Services/SurfaceTrackingJobService.cs</code>	 Hangfire tracking job entry point
<code>dotnet-api/Services/SurfaceDetectionPipeline.cs</code>	Unified pipeline orchestrator (Gemini → SAM3)
<code>dotnet-api/Services/SceneDetectionJobService.cs</code>	Hangfire detection job entry points
<code>dotnet-api/Services/RenderJobService.cs</code>	 Real — FFmpeg compositing + libx264 encode
<code>dotnet-api/Services/PlatformSettingsService.cs</code>	DB-backed settings with appsettings fallback
<code>dotnet-api/Program.cs</code>	DI registration of all engines (4 slots + 17 engine classes)
<code>detection-service/main.py</code>	Python FastAPI server
<code>detection-service/detector.py</code>	YOLO detector + ByteTrack
<code>detection-service/engine_v2.py</code>	v2 pipeline orchestrator
<code>detection-service/grounding_dino_detector.py</code>	Grounding DINO detector
<code>detection-service/sam_segmenter.py</code>	SAM segmenter
<code>detection-service/depth_estimator.py</code>	Depth Anything V2
<code>detection-service/brand_safety.py</code>	CLIP brand-safety
<code>detection-service/tracker.py</code>	Surface tracker

13. Governance Compliance

Rule	Status
No mock code (<code>governance/rules/no-mock-code.md</code>)	 <code>Basic*Service</code> classes throw to force real engine config
Real implementations only	 All 5 detection engines + 2 brand analysis engines + compositing + render + tracking are real API calls or local inference
Cross-stack completeness (<code>governance/rules/file-ownership.md</code>)	 DI → interface → implementation → controller → frontend types
Verification (<code>governance/rules/verification.md</code>)	 All facts verified from actual source files (2026-07-25)

Rule	Status
Prerequisites (governance/rules/prerequisites.md)	✅ Feature files + NFRs + plans for all phases
Contract freshness	✅ Migration AddSurfaceTrackingBoundaries created; API contract updated

Last Updated: 2026-07-25 (v3.0 — post Phase 0-3 engine upgrade)

Maintained by: BIT Platform Engineering